

Q&A poznámky k obhajobe

Toto nie je hlavný skript. Je to technická príprava na otázky k práci. Držať sa téz z práce: Merkle–Hellman, jeho slabiny, varianty, implementácia a experimenty. Nepúšťať sa do dôkazov, ktoré v práci nie sú rozpracované.

1. Prečo NP-úplnosť subset-sum nestačí?

NP-úplnosť je worst-case tvrdenie pre všeobecný rozhodovací problém. Kryptosystém však negeneruje ľubovoľné inštancie. Generuje štruktúrované verejné kľúče, ktoré musia zostať ľahké pre držiteľa súkromného kľúča.

Decision subset-sum:

$$\exists x \in \{0, 1\}^n : \sum_{i=1}^n x_i w_i = S$$

V kryptografii útočníka zaujíma search verzia: nájsť konkrétny vektor x , lebo ten reprezentuje správu.

Dôležitá veta z práce: komplexita všeobecného problému nedáva záruku o náročnosti konkrétnej knapsack inštancie. Pri Merkle–Hellmanovi verejný kľúč vzniká z ľahkej superrastúcej postupnosti cez modulárnu transformáciu, teda nejde o náhodnú všeobecnú inštanciu.

2. Korektnosť Merkle–Hellmanovej schémy

Súkromná postupnosť w je superrastúca. Volíme $m > \sum_i w_i$ a $\gcd(r, m) = 1$. Verejné váhy:

$$b_i \equiv r w_i \pmod{m}$$

Šifrovanie:

$$C = \sum_{i=1}^n x_i b_i$$

Dešifrovanie použije r^{-1} :

$$C' \equiv r^{-1} C \pmod{m}$$

Keďže $b_i \equiv r w_i \pmod{m}$:

$$C' \equiv \sum_{i=1}^n x_i w_i \pmod{m}$$

Podmienka $m > \sum_i w_i$ zabezpečí, že každý možný súkromný subset-sum je menší než m . Modulárna redukcia teda nestratí informáciu:

$$C' = \sum_{i=1}^n x_i w_i$$

Potom greedy algoritmus nad superrastúcou postupnosťou obnoví bity. Podmienka $\gcd(r, m) = 1$ je presne preto, aby existovala modulárna inverzia r^{-1} .

Ak sa spýtajú na jednoznačnosť: pri superrastúcej postupnosti má súčet jednoznačnú reprezentáciu, lebo najväčší použiteľný prvok nemôže byť nahradený súčtom menších.

3. Shamirov útok — čo je jadro

Najprv oprava intuície: útočník nepozná u , m , w_i , π ani k_i . Pozná však celý verejný kľúč, teda všetky váhy b_i . Útok sa snaží z týchto verejných čísel získať ekvivalentný trapdoor.

Verejné váhy vznikli takto:

$$b_i \equiv r w_{\pi(i)} \pmod{m}$$

Označme $u \equiv r^{-1} \pmod{m}$. Keby sme poznali u a m , spätný prevod by bol:

$$ub_i \equiv w_{\pi(i)} \pmod{m}$$

To isté bez kongruencie znamená, že existuje celé číslo k_i :

$$ub_i - k_i m = w_{\pi(i)}$$

Číslo k_i je len pomocná neznáma: počet násobkov m , ktoré by sa odčítali z ub_i , aby zostal zvyšok $w_{\pi(i)}$. Útočník ho priamo nepozná.

Vydelením mb_i dostaneme:

$$\frac{u}{m} - \frac{k_i}{b_i} = \frac{w_{\pi(i)}}{mb_i}$$

Teraz označme chybu:

$$\varepsilon_i = \frac{w_{\pi(i)}}{mb_i}$$

Potom:

$$\frac{k_i}{b_i} = \frac{u}{m} - \varepsilon_i$$

Pre indexy zodpovedajúce malým súkromným váham je ε_i malé. Teda viaceré zlomky $\frac{k_i}{b_i}$ sú blízko tomu istému neznámemu číslu:

$$\frac{k_i}{b_i} \approx \frac{u}{m}$$

$$\frac{k_j}{b_j} \approx \frac{u}{m}$$

Preto sú blízko aj medzi sebou:

$$\frac{k_i}{b_i} \approx \frac{k_j}{b_j}$$

Po krížovom násobení:

$$k_i b_j \approx k_j b_i$$

čiže

$$|k_i b_j - k_j b_i| < E$$

Toto je miesto, kde sa objaví informácia z verejného kľúča: b_i a b_j sú známe verejné čísla. Neznáme sú celé čísla k_i, k_j , ale musia spĺňať malé celočíselné vzťahy s verejnými koeficientmi.

Shamirov výsledok je, že pre pevný počet vhodne vybraných indexov sa takéto celé hodnoty dajú nájsť v polynomiálnom čase pomocou celočíselného programovania v pevnej dimenzii. Ak je použitá permutácia, útočník skúša pevný počet kandidátnych indexov; to je stále len polynomiálne veľa možností.

Z nájdenej k_i sa zlomky $\frac{k_i}{b_i}$ použijú ako aproximácie pomeru $\frac{u}{m}$. Z tejto aproximácie sa skonštruuje iná modulárna transformácia, ktorá prevedie verejné váhy na ľahko riešiteľnú postupnosť.

Ako to povedať stručne: Shamir nehľadá správu a nemusí obnoviť pôvodný súkromný kľúč. Hľadá malé celočíselné vzťahy medzi verejnými váhami, ktoré prezrádzajú ekvivalentný trapdoor.

4. Low-density útoky a LLL embedding

Hustota verejnej subset-sum inštancie:

$$d = \frac{n}{\log_2} \left(\max_i b_i \right)$$

Nízka hustota znamená, že váhy sú veľké vzhľadom na počet prvkov. Možné subset-sumy sú redšie a správne riešenie môže vystúpiť ako krátky vektor v mriežke.

Máme ciphertext:

$$C = \sum_{i=1}^n x_i b_i, \quad x_i \in \{0, 1\}$$

Použijeme vektory:

$$v_i = (0, \dots, 1, \dots, 0, -b_i)$$

$$v_{n+1} = (0, \dots, 0, C)$$

Ak x je riešenie, potom:

$$\sum_{i=1}^n x_i v_i + v_{n+1} = \left(x_1, \dots, x_n, C - \sum_{i=1}^n x_i b_i \right) = (x_1, \dots, x_n, 0)$$

Tento vektor je krátky, lebo má len nuly a jednotky v prvých súradniciach a nulu na konci. LLL sa snaží nahradiť bázu mriežky kratšími a viac ortogonálnymi vektormi; pri nízkej hustote často odhalí práve vektor zodpovedajúci riešeniu.

V práci uvádzaná hranica: Lagarias–Odlyzko analyzovali náhodné subset-sum inštancie s hustotou približne pod 0.645, kde je cieľový vektor s vysokou pravdepodobnosťou najkratší nenulový vektor vhodnej mriežky. Toto nie je univerzálna bezpečnostná hranica.

Dôležité rozlíšenie: LLL/low-density útok nehľadá trapdoor. Hľadá priamo riešenie verejnej subset-sum inštancie.

5. Brickellov útok na iterované batohy

Iterovaný Merkle–Hellman používa viac vrstiev modulárnej transformácie:

$$a_i^{(j)} \equiv r_j a_i^{(j-1)} \pmod{m_j}, \quad j = 1, \dots, Y$$

Dešifrovanie ide opačným smerom cez inverzie jednotlivých vrstiev až k pôvodnej superrastúcej postupnosti.

Cieľ iterácie: lepšie skryť pôvodný trapdoor. Problém: stále ide o postupnosť štruktúrovaných modulárnych relácií. Brickellov útok používa lattice basis reduction na vybrané podmnožiny verejných váh. Hľadajú sa krátke vektory, ktoré vznikajú zo skrytých relácií medzi vrstvami; Brickell ich označuje ako desirable vectors.

Z týchto vektorov sa dá obnoviť dosť informácie o skrytej štruktúre na získanie postupnosti, ktorá sa správa ako superrastúca pre väčšinu pozícií. Zvyšné pozície sa riešia samostatne. Netreba presný pôvodný kľúč, stačí ekvivalentná dešifrovacia štruktúra.

V práci: Brickell reportoval úspešné útoky pre $n = 100$ a $Y = 5, 10, 20$. Záver: iterácia neopravila Merkle–Hellmanovu myšlienku, len zmenila formu slabiny.

6. Chor–Rivest – konštrukcia a dešifrovanie

Chor–Rivest je knapsack-type systém, ale nie je jednoduchá úprava Merkle–Hellmana. Nepoužíva superrastúcu postupnosť. Trapdoor je v aritmetike konečného poľa.

Parametre: p, h a pole $\text{GF}(p^h)$. Plaintext blok je vektor dĺžky p s presne h jednotkami:

$$x = (x_0, \dots, x_{p-1}), \quad \sum_i x_i = h$$

Počet platných blokov je $\binom{p}{h}$, takže jeden blok vie niesť najviac:

$$\left\lfloor \log_2 \binom{p}{h} \right\rfloor$$

bitov informácie.

Vyberie sa prvok $t \in \text{GF}(p^h)$ stupňa h nad $\text{GF}(p)$ a generátor g multiplikatívnej grupy. Pre každé $\alpha \in \text{GF}(p)$:

$$g^{a_\alpha} = t + \alpha$$

Verejné váhy vzniknú permutáciou a offsetom:

$$c_i \equiv a_{\pi(i)} + d \bmod (p^h - 1)$$

Šifrovanie:

$$S = \sum_{i=0}^{p-1} x_i c_i$$

Keďže plaintext má presne h jednotiek, offset prispieva presne hd . Po odčítaní:

$$g^{S-hd} = \prod_{i:x_i=1} (t + \pi(i))$$

Držiteľ kľúča interpretuje výsledok ako polynóm nad $\text{GF}(p)$ a faktorizuje ho. Faktory ukážu, ktoré členy $t + \pi(i)$ sa v súčine nachádzali, teda ktoré pozície správy boli jednotky.

Najkratšie: Chor–Rivest zachováva verejný knapsack tvar, ale trapdoor nie je greedy superrastúca postupnosť; je to faktorizácia v konečnom poli po prevode cez diskkrétne logaritmy.

7. Vaudenayov útok na Chor–Rivest

Chor–Rivest sa snažil vyhnúť nízkej hustote a slabine superrastúceho trapdooru. Napríklad pre parametre $p = 197$, $h = 24$ má hustotu približne 0.93.

Útok Vaudenaya nie je low-density útok. Využíva algebraickú štruktúru verejného kľúča:

$$c_i \equiv a_{\pi(i)} + d \bmod (p^h - 1)$$

pričom

$$g^{a_\alpha} = t + \alpha$$

Teda verejné váhy nie sú náhodné subset-sum váhy. Sú odvodené z diskrétnych logaritmov špeciálnych prvkov tvaru $t + \alpha$, následne skryté permutáciou a offsetom.

V práci: Vaudenay ukázal útoky na navrhnuté parametre vrátane $\text{GF}(197^{24})$, $\text{GF}(211^{24})$, $\text{GF}(243^{24})$ a $\text{GF}(256^{25})$. Pre $\text{GF}(197^{24})$ odhadoval asi 2^{29} operácií v $\text{GF}(197)$ a prakticky obnovil tajný kľúč za približne 15 minút na starom notebooku.

Bezpečná odpoveď: Chor–Rivest odstránil priamu Merkle–Hellmanovu slabinu, ale verejný kľúč stále niesol špeciálnu algebraickú štruktúru. Tú Vaudenay využil.

8. Implementácia: čo je vlastné a čo sa meralo

Implementácia je experimentálny nástroj, nie bezpečná knižnica. Varianty: mh-classic, mh-permuted, mh-iterated.

Spoločné rozhranie SchemeOps: keygen, encrypt, decrypt, cleanup. Veľké čísla cez GMP (mpz_t). Randomizované časti cez PCG; bez explicitného seedu sa seed berie z OS entropie, pri experimentoch sa používa explicitný seed kvôli reprodukovateľnosti.

Generovanie súkromnej postupnosti v implementácii:

$$w_i = S + \delta_i, \quad \delta_i \in \{1, \dots, \Delta\}$$

kde S je súčet predchádzajúcich váh a $\Delta = \text{MH_DEFAULT_DELTA_MAX}$.

Modul:

$$m = S + \mu, \quad \mu \in \{1, \dots, Fn\}$$

kde $F = \text{MH_DEFAULT_MARGIN_FACTOR}$.

V základnej konfigurácii: $\Delta = 64$, $F = 64$.

Benchmarks: $n \in \{128, 256, 512, 1024, 2048, 4096\}$, tri deterministické seedy, päť meraných opakovaní po troch warm-up opakovaní. Pri benchmarku sa po encrypt vykoná decrypt a bity sa porovnávajú s pôvodným vstupom.

Výsledky z práce:

- pri $n = 4096$: classic približne 11.057 ms, iterated približne 30.929 ms,
- pri classic $n = 4096$: keygen približne 10.895 ms, encrypt 0.089 ms, decrypt 0.072 ms,
- iterované vrstvy: pri $n = 4096$ dve vrstvy 30.634 ms, päť vrstiev 88.693 ms,
- density pri $n = 128$: dve vrstvy 0.916, päť vrstiev 0.812,
- delta sweep pri $n = 128$: $\Delta = 1$ dá density 1.000, $\Delta = 128$ približne 0.948.

9. Solver experimenty verus kryptanalýza

Implemented attack mode rieši priamy subset-sum nad verejnými váhami a ciphertextom. Nie je to implementácia Shamira, LLL ani Brickella.

Brute force skúša všetky vektory:

$$O(2^n)$$

Meet-in-the-middle rozdelí váhy na dve polovice. Vytvorí tabuľky čiastočných súčtov a hľadá dvojice:

$$2^{\lfloor \frac{n}{2} \rfloor} + 2^{\lceil \frac{n}{2} \rceil}$$

Časovo približne $O(2^{\frac{n}{2}})$, pamäťovo tiež exponenciálne. V práci: brute force testovaný do $n = 22$ s časom okolo 188 ms; MITM do $n = 40$ s časom okolo 757 ms. Pri $n = 40$ má tabuľka 2097152 záznamov.

Ak sa spýtajú, prečo som neimplementoval Shamira/LLL: implementované solvery sú vzdelávací baseline pre priame hľadanie správy. Shamir a LLL sú štrukturálne útoky a v práci sú spracované teoreticky.

10. Moderný kontext a RSA

Subset-sum z kryptografie nezmizol. Moderný výskum ho používa skôr ako explicitný hardness assumption, nie ako skrytú superrastúcu štruktúru. V práci sú spomenuté napríklad práce Impagliazzo–Naor a neskoršie schémy/redukcie založené na random subset-sum.

Klasické trapdoor knapsack systémy zlyhali preto, že verejné kľúče neboli náhodné hard inštancie. Boli generované zo špeciálnej súkromnej štruktúry.

RSA je dobrý kontrast: RSA funkcia je

$$x \rightarrow x^e \bmod N$$

Súkromná informácia súvisí s faktorizáciou $N = pq$. Faktorizácia je v NP, ale nie je známe, že by bola NP-úplná, a všeobecne sa to neočakáva. Bezpečnosť RSA teda nie je argument „máme NP-úplný problém“, ale konkrétny predpoklad o náročnosti inverzie RSA funkcie / faktorizácie veľkých čísel.

Záver k práci: ťažký všeobecný problém nestačí. Rozhoduje konkrétna konštrukcia, generovanie kľúčov a štruktúra verejných inštancií.

Mini backup: ak padnú okrajové témy z práce

- Multiplicative knapsack: Merkle–Hellman spomínali aj trapdoor cez produkty relatívne prvočíselných čísel; Odlyzko uvádza útoky na praktické parametre.
- Knapsack podpisy/autentifikácia: Shamir fast signature a Schöbi–Massey boli tiež napadnuté lattice-based metódami.
- Regular private weights: príliš pravidelný rast môže prezradiť relácie typu $b_{j+1} = 2b_j$ alebo $b_{j+1} = 2b_j - m$, z čoho sa môže objavovať skrytý modul.
- PCG: vhodné pre reprodukovateľné experimenty, nie tvrdenie o produkčnej kryptografickej náhodnosti.